

E-Portfolio als Modulprüfung WiSe25-26 Künstliche Intelligenz

Thema: CNN für den MNIST-Datensatz

Betrachten Sie den MNIST-Datensatz zur Handschriftenerkennung von Ziffern, wie in der Vorlesung behandelt. Entwickeln Sie ein Convolutional Neural Network (CNN) und vergleichen Sie Ihr Modell mit den in der Vorlesung behandelten Netzwerken.

Verfasst und redigiert von:	Gilles Armel Takoufouet Koudjou
Matrikelnummer:	8216595
Studiengang:	Master Data Science
Gruppennummer:	114
Übrige Gruppenmitglieder:	Venja Pape, Max Wagner, Dung Nguyen
Sprecherin:	Venja Pape

Zusammenfassung des Logbuchs

Im Rahmen des Projekts untersuchte jedes Gruppenmitglied bestimmte Aspekte des Convolutional Neural Networks.

Mein eigener Arbeitsbereich umfasste vor allem die Optimierung architektonischer und trainingsbezogener Parameter, Fragen der Regularisierung sowie die Analyse der Robustheit gegenüber modifizierten Daten. Ziel war es, zentrale Bestandteile des CNN systematisch zu optimieren, seine Generalisierungsfähigkeit zu verbessern und dabei die Modellkomplexität in einem vertretbaren Rahmen zu halten.

I-Hyperparametersoptimierung

Im Rahmen der Hyperparameteroptimierung wurden verschiedene Modellkonfigurationen getestet. Dabei wurden unter anderen die Anzahl der Filter, die Aktivierungsfunktion, die Anzahl der Convolution-Layer, die Anzahl der Dense-Layer sowie die Dropout-Rate variiert.

1) Aktivierungsfunktionen

Als erster Schritt der Hyperparameteroptimierung wurden die Aktivierungsfunktionen untersucht, da sie einen direkten Einfluss auf die Nichtlinearität des Modells und somit auf dessen Lernfähigkeit haben. Zusätzlich zu „standard“ Aktivierungsfunktionen wurden deren Kombinationen (z. B. ReLU + Tanh) betrachtet, um potenzielle komplementäre Effekte auszunutzen, wie etwa stabile Gradienten (ReLU) und feinere Wertebereiche (Tanh).

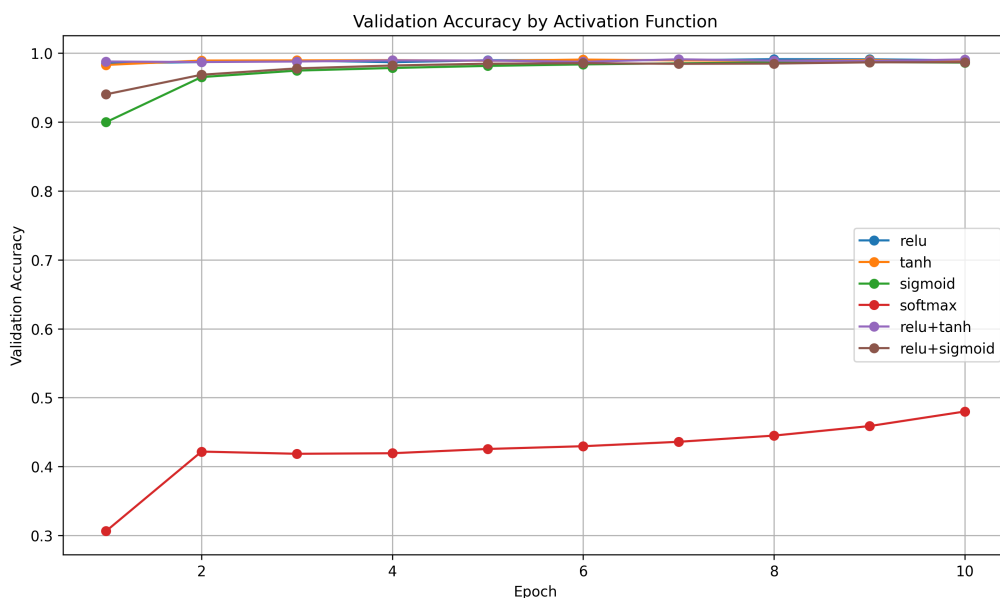


Abbildung 1: Vergleich der Aktivierungsfunktionen

Die Ergebnisse zeigen, dass ReLU-basierte Modelle (insbesondere ReLU und ReLU+Tanh) die höchste Validierungsgenauigkeit erreichen und zudem schnell konvergieren. Die Softmax-Aktivierung als versteckte Aktivierungsfunktion führt zu sehr schlechten Ergebnissen, was

ihre ungeeignete Verwendung außerhalb der Ausgabeschicht bestätigt. Die [Tabelle 4](#) zeigt die Accuracy und andere Statistiken jeder Funktion.

2) Anzahl der Convolutional Layers

Im nächsten Schritt wurden mit den zwei besten Aktivierungsfunktionen (ReLU und ReLU + Tanh) leichte Modelle gebaut und die Architektur des Netzwerks genauer untersucht, insbesondere die Anzahl der Convolution-Layers. Ziel war es zu analysieren, ob eine tiefere Architektur zu einer besseren Extraktion von Merkmalen und somit zu einer höheren Klassifikationsgenauigkeit führt. Getestet wurde die `conv_blocks_list` [1,2,3] und aus den Ergebnissen lässt sich das Fazit ziehen: Je tiefer ein Modell ist, desto höher ist seine Leistung.

Ebenfalls wurde visuell dargestellt, wie sich nach Epochen und Convolutional Blocks die *val_accuracy* veränderte (siehe [Abbildung 3](#)).

3) Anzahl der Filters

Nachdem der Einfluss der Anzahl der Convolution-Layer untersucht wurde, stellte sich die Frage, wie viele Filter innerhalb dieser Layer verwendet werden sollten, da die Anzahl der Filters die Breite und damit die Kapazität zur Merkmalsextraktion auf jeder Ebene beeinflusst. Die Base Filter 64 hat den besten Score, jedoch braucht zu viel Parameters.

Außerdem ergibt sich aus der Analyse der Filters ([Abbildung 6](#), [Abbildung 5](#), [Tabelle 7](#)), dass die Filter der ersten Convolution-Schicht hauptsächlich Kanten und Konturen der Ziffer extrahieren, was typisch für frühe CNN-Schichten ist. Die statistischen Werte (nahe Null liegender Mittelwert und moderate Standardabweichung) deuten darauf hin, dass die Gewichte gut initialisiert und stabil trainiert wurden, ohne extreme Ausreißer.

4) Verborgene Dense Layers, Learning Rate, Dropout Rate

Zweck der Untersuchung hier war einerseits herauszufinden, ob eine höhere Anzahl von Dense Layers die Klassifikationsentscheidung verbessern würde, andererseits, welche Rate Overfitting vermeiden kann. Die genutzte Methode war die Grid Search, um alle möglichen Konfigurationen zu testen. [Tabelle 8](#) zeigt die Top 10, und alle diese Modelle haben zumindest zwei Dense Layers. Außerdem führen extrem große oder kleine Learning Rates (10^{-2} und 10^{-4}) zu keinen guten Ergebnissen. Das größte Problem hier war die Laufzeit (mehr als 20 Stunden).

Top 10 CNN Models by Validation Accuracy

filters	activation	conv_blocks	dense_layers	dropout	learning_rate	train_loss	train_accuracy	val_loss	val_accuracy	params
64	relu	3	2	0.25	0.001	0.0087	0.9978	0.0455	0.9921	675370
32	tanh	3	2	0.5	0.001	0.0076	0.9974	0.041	0.9914	249162
32	relu+tanh	3	2	0.25	0.001	0.0081	0.9978	0.0349	0.9911	249162
32	relu	2	1	0.25	0.001	0.0046	0.9986	0.0341	0.991	673610
64	relu+tanh	3	3	0.5	0.001	0.0088	0.9972	0.0472	0.9907	673610
64	relu	3	2	0.5	0.001	0.0101	0.9968	0.0379	0.9907	673610
32	relu	3	3	0.25	0.001	0.0144	0.996	0.0421	0.9906	250922
32	relu	3	3	0.0	0.001	0.0095	0.997	0.0415	0.9904	250922
32	relu+tanh	3	3	0.25	0.001	0.0134	0.9961	0.0464	0.9904	250922
64	relu	3	3	0.5	0.001	0.0159	0.996	0.0609	0.9904	675370

Tabelle 1: Top 10 Modelle der Grid Search

5) Dropout alleine und Tuning

Da aus der vorherigen Grid Search keine klare Schlussfolgerung über den Dropout gezogen werden konnte, wurde anschließend dessen Einfluss isoliert betrachtet. Dabei wurden alle anderen Modellparameter fixiert. Aus [Tabelle 9](#) stellte sich fest, dass die Leistung des Modells ohne Dropout (0.00) am schwächsten ist.

Aufgrund der zu langen Laufzeit der normalen Grid Search wurde eine Tuning-Strategie in Betracht gezogen, um den Suchprozess effizienter zu gestalten. Diesbezüglich wurde der Keras Tuner genutzt. Die genutzten Parameters waren genau dieselben wie bei der Grid Search, jedoch wurden unterschiedliche Ergebnisse erreicht. Da die Accuracy des besten Modells hier schon kleiner als die des zehnten Modells der Grid Search war, kann man davon ausgehen, dass das beste Modell des Tuning nur ein lokales Maximum ist. Deswegen ist im weiteren Verlauf das Tuning nicht mehr berücksichtigt worden.

II-Weitere Parameters

1) Batch Normalisation und Batch Size

Die Batch Normalization ist ein Prozess, der das Training durch Normierung der Aktivierungen stabilisiert. Sie sollte auf die Frage antworten, ob die Performance des Modells noch durch diese zusätzliche Stabilisierung des Trainingsprozesses verbessert werden kann. Dabei mussten vor der Nutzung `model_1` und `model_3` ([Tabelle 15](#)) neu trainiert werden, da sich dadurch die Verteilung der Daten im Netzwerk ändert. Die Ergebnisse der [Tabelle 11](#) zeigen, dass die Batch Normalization keinen signifikanten Einfluss auf die Modellleistung hat. Die Unsicherheit war, ob die Senkung der Scores daran liegt, dass die beim neuen Training genutzten Daten unterschiedlich waren, oder einfach nur an der Batch Normalization.

Die Batch Size wurde ebenfalls auf neu trainiertem `model_1` ([Tabelle 15](#)) untersucht und in der [Tabelle 12](#) dokumentiert. Die kleine Size hat die beste Accuracy, aber verlangt mehr Zeit gekostet. Die Size 128 scheint ein guter Kompromiss zu sein.

2) Dense Units

Die Anzahl der Dense Units beeinflusst dabei direkt die Modellkapazität. Eine zu geringe Anzahl kann dazu führen, dass das Modell nicht in der Lage ist, komplexe Zusammenhänge zu erfassen (Underfitting), während eine zu hohe Anzahl die Gefahr von Overfitting erhöht.

Ziel dieser Untersuchung war es daher, ein geeignetes Gleichgewicht zwischen Modellkomplexität und Generalisierungsfähigkeit zu finden. Das `model_1` wurde zu diesem Zweck genutzt und 128 war die zu nutzende Anzahl von Units (siehe [Tabelle 13](#)).

3) Optimizer Adam vs SGD

Anschließend wurde der Einfluss des Optimizers untersucht, indem Adam und SGD (Stochastic Gradient Descent) direkt miteinander verglichen wurden. Der Optimizer SGD scheint beim Training eine bessere Accuracy zu haben, diese ist jedoch bei der Validierung abgestiegen. Die vernünftige Entscheidung wäre daher, beim Adam Optimizer zu bleiben ([Tabelle 14](#)).

III-Blick auf die modifizierten Daten

Die vier Modelle wurden zusätzlich auf modifizierten Daten validiert, um ihre Robustheit gegenüber gezielten Veränderungen der Eingabedaten zu überprüfen. In diesem Abschnitt lag der Schwerpunkt nicht auf dem Verständnis oder der eingehenden Interpretation der Reaktionen der Modelle, da dies Gegenstand der Studie der Sprecherin war.

1) Validierung auf modifizierten Daten

Die Modelle wurden auf gaußischem Rauschen, Rotierung, Pepper und Salz, dickeren Zahlen, Verschiebung, Speckled Rauschen, Kombination Pepper und Salz + gaußischem Rauschen und Okklusion validiert. Die Tabellen [16](#), [17](#), [18](#), [19](#), [20](#) und [21](#) dokumentieren und veranschaulichen jeweils die erzielten Ergebnisse in ausreichender Weise. Nachstehend werden dennoch die Ergebnisse der Okklusion dargestellt, bei der die Modelle die schlechtesten Ergebnisse erzielten.

Validation Results on Occlusion

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_4	32	relu	2	1	0.2500	0.0010	0.2691	0.9274
model_1	64	relu	3	2	0.2500	0.0010	0.3324	0.9244
model_2	64	tanh	3	2	0.5000	0.0010	0.2882	0.9244
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.3377	0.9100

Tabelle 2: Ergebnisse bei Okklusion

2) Retraining auf den modifizierten Daten

Die vier Modelle wurden des Weiteren auf einer Mischung von normalen und modifizierten Daten trainiert. Die folgende Grafik veranschaulicht die Robustheit der vier Modelle vor und nach dem Retraining insbesondere unter gaußischem + Pepper-und-Salz-Rauschen, wobei eine riesige Verbesserung zu sehen ist (siehe [Abbildung 13](#)).

IV- Zwischenstand und weiterführende Arbeiten

Zum Schluss der Arbeit wurden die Modelle auf die Testdaten angewendet und in der [Tabelle 24](#) dokumentiert. Die nächsten Schritte wären das Retraining auf weiteren Modifikationen

sowie auf anderen Datensätzen (z. B. EMNIST).

Test Results of Selected Models

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	test_loss	test_accuracy	mean_confidence
model_1	64	relu	3	2	0.25	0.001	0.0246	0.9928	0.9959
model_4	32	relu	2	1	0.25	0.001	0.0285	0.9913	0.9933
model_2	64	tanh	3	2	0.5	0.001	0.0334	0.9907	0.9953
model_3	32	relu+tanh	3	2	0.25	0.001	0.0377	0.9882	0.9913

Tabelle 3: Testergebnisse auf den Testdaten

Einschätzung der Gruppenmitglieder

Venya

Stärken:

- **Hörsamkeit:** Sie war während der Treffen offen für Kritik und Vorschläge. Ihr ruhiges Verhalten half, alle Ideen im Prozess in Betracht zu ziehen.

Schwächen:

- **Selbstbewusstsein:** Manchmal zeigte sie große Unsicherheiten bezüglich ihres Arbeitsteils, was die Stärke ihrer Ergebnisse abschwächen konnte.

Max

Stärken:

- **Kreativität:** Er hat ein tiefes Verständnis der Prinzipien, die in der Lösung angewendet wurden. Schon am Anfang hatte er viele Ideen, wie das Projekt gegliedert werden konnte.

Schwächen:

- **Ungeduld:** Er wollte oft sehr schnell vorankommen, sodass wir manchmal nicht auf derselben Wellenlänge waren.

Dung

Stärken:

- **Neugier:** Er hat während der ganzen Arbeit Fragen zu vielen Details gestellt, um ein gutes Verständnis zu haben. Das hat auch manchmal zum Auftauchen anderer Ideen geführt.

Schwächen:

- **Zurückhaltung:** Er wirkte manchmal eher zurückhaltend, was den direkten Austausch über unsere Ergebnisse erschwerte, insbesondere vor dem Hintergrund, dass wir manche ähnliche Untersuchungen durchgeführt haben.

Individuelles Logbuch

22.02.2026: Erste EDA und Zusammensetzung mit dem Datensatz

- EDA
- Analyse der Fehlklassifikationen

29.03.2026: Hyperparametersoptimierung Genesis

Nach zwei ersten Gruppentreffen (dokumentiert durch die Sprecherin) wurde beschlossen, dass jedes Gruppenmitglied einen spezifischen Teil des Convolutional Neural Networks untersucht. Ziel war es, die einzelnen Ergebnisse später zu kombinieren, um ein optimales Gesamtmodell zu entwickeln. In diesem Zusammenhang bestand meine Aufgabe zunächst darin, die Hyperparameter sowie die Filterkonfigurationen zu analysieren und zu optimieren.

Aktivierungsfunktionen

- Fragestellung/Begründung:

Einfluss verschiedener Aktivierungsfunktionen analysieren, um ein grundlegendes Verständnis für die nichtlinearen Transformationen im Netzwerk zu gewinnen. Ziel war es zu untersuchen, ob die Kombination der beiden Funktionen Vorteile gegenüber einzelnen Aktivierungsfunktionen bietet.

- Vorgehen:

Unterschiedliche Funktionen testen:

- Softmax
- ReLU
- Tanh
- Sigmoid
- ReLU + Tanh
- ReLU + Softmax

- Ergebnisse:

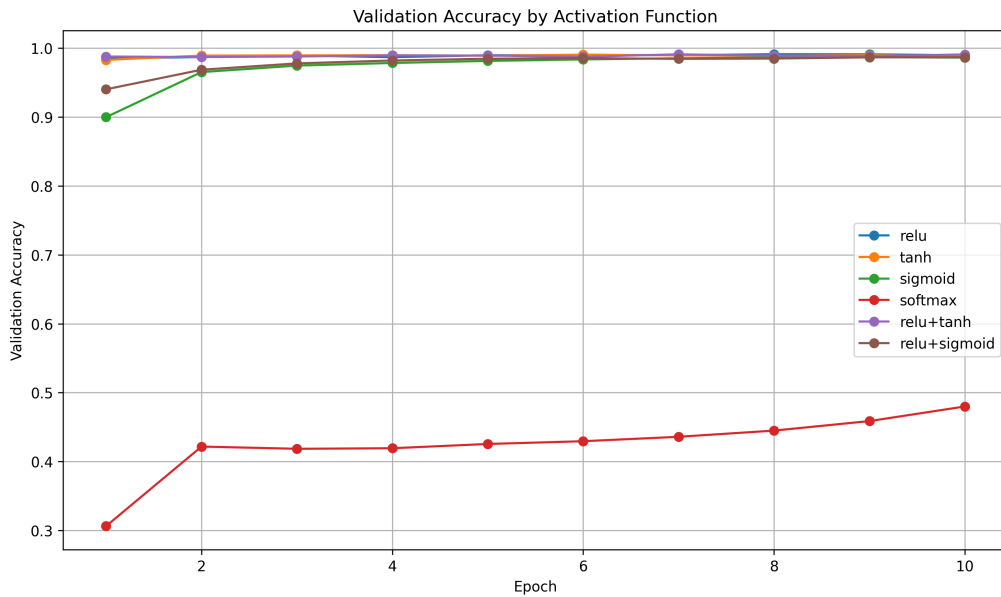


Abbildung 2: Graphische Ergebnisse der Aktivierungsfunktionen

activation	val_loss	val_accuracy	test_loss	test_accuracy	params
relu+tanh	0.052830	0.9908	0.039232	0.9895	421642
relu	0.046030	0.9896	0.042952	0.9883	421642
tanh	0.055293	0.9882	0.041350	0.9870	421642
relu+sigmoid	0.060964	0.9868	0.055707	0.9839	421642
sigmoid	0.049470	0.9862	0.050223	0.9832	421642
softmax	1.901045	0.4800	1.886699	0.4715	421642

Tabelle 4: Tabelle der Aktivierungsfunktionen

Die Funktion Softmax ist nicht angenehm für die Hidden Layers, da sie den schlimmsten Score hat.

ReLU, Tanh, ReLU+Tanh sind mit kleinen Abständen Spitzenreiter.

Anzahl der Convolutional Layers

- Fragestellung/Begründung:

Ziel war es zu analysieren, ob eine tiefere Architektur zu einer besseren Extraktion von Merkmalen und somit zu einer höheren Klassifikationsgenauigkeit führt.

- Vorgehen:

Leichte Modelle mit den vorherigen besten Aktivierungsfunktionen (ReLU, ReLU+Tanh) wurden gebaut und getestet wurde mithilfe einer Schleife die `conv_blocks_list` [1,2,3].

Im Fall von mehr als einem Block ist die Struktur wie folgt:

- Conv2D_1 (32)

- Conv2D_2 (64)
- Conv2D_3 (128)

- Ergebnisse:

Die zwei besten Modelle haben 3 Convolutional Layers und das dritte hat 2.

Fazit: Je tiefer ein Modell ist, desto höher ist seine Leistung.

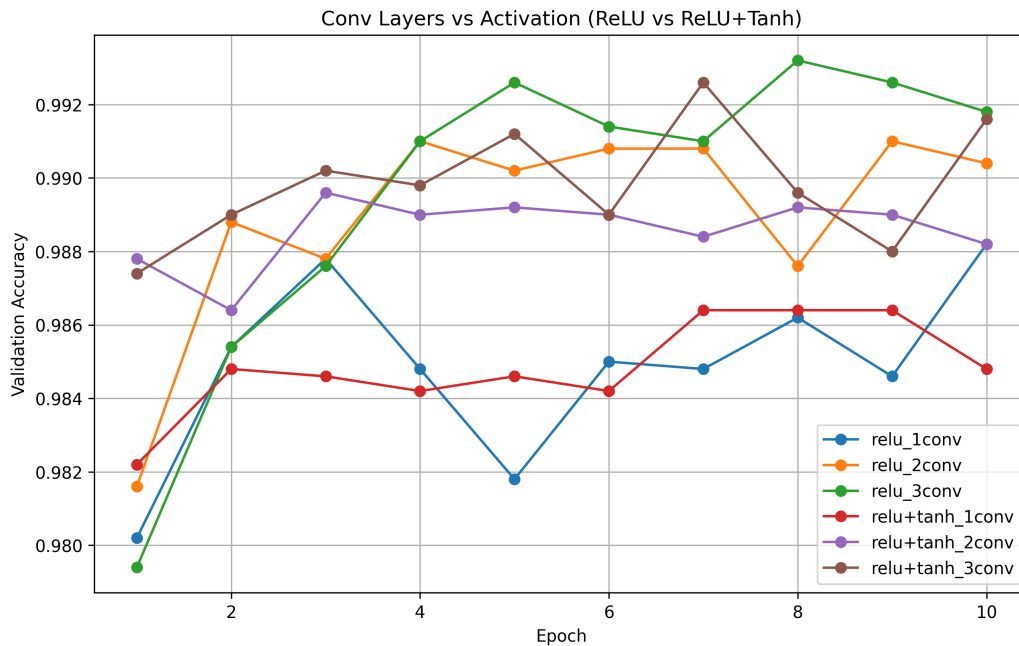


Abbildung 3: Verlauf der Validierungsgenauigkeit für verschiedene Convolutional Blocks

Conv Layers Results (Best Model Highlighted)

activation	conv_layers	val_loss	val_accuracy	params
relu	3	0.0391	0.9918	241,546
relu+tanh	3	0.0519	0.9916	241,546
relu	2	0.0448	0.9904	421,642
relu	1	0.0561	0.9882	804,554
relu+tanh	2	0.0663	0.9882	421,642
relu+tanh	1	0.0721	0.9848	804,554

Tabelle 5: Tabellarische Ergebnisse zu den Convolutional Blocks

- Grenzen

Ich wollte ebenfalls 5, 10 und 30 Convolutional Layers testen, aber ich habe festgestellt, dass es nicht möglich wäre, weil die Bilder aus dem Format 28 sind und nach jedem Convolution Block diese Zahl halbiert wird. Also ist in diesem Sinne 3 die maximale Anzahl.

Filters und deren Analyse

- Fragestellung:

Durch die Erhöhung der Anzahl der Filter wird die Merkmalsextraktion verbessert und die Validierungsgenauigkeit erhöht?

- Vorgehensweise:

Nutzung von drei leichten Modellen mit jeweils den Aktivierungsfunktionen ReLU, Tanh und ReLU + Tanh.

Die Convolutional-Zahl ist auf 3 fixiert, weil sie vorher die besten Ergebnisse erzielt hat.

Getestet wurde `filters_list` (16, 32, 64).

- Ergebnisse:

Die Filters 64 haben am meisten die besten Ergebnisse vorgezeigt.

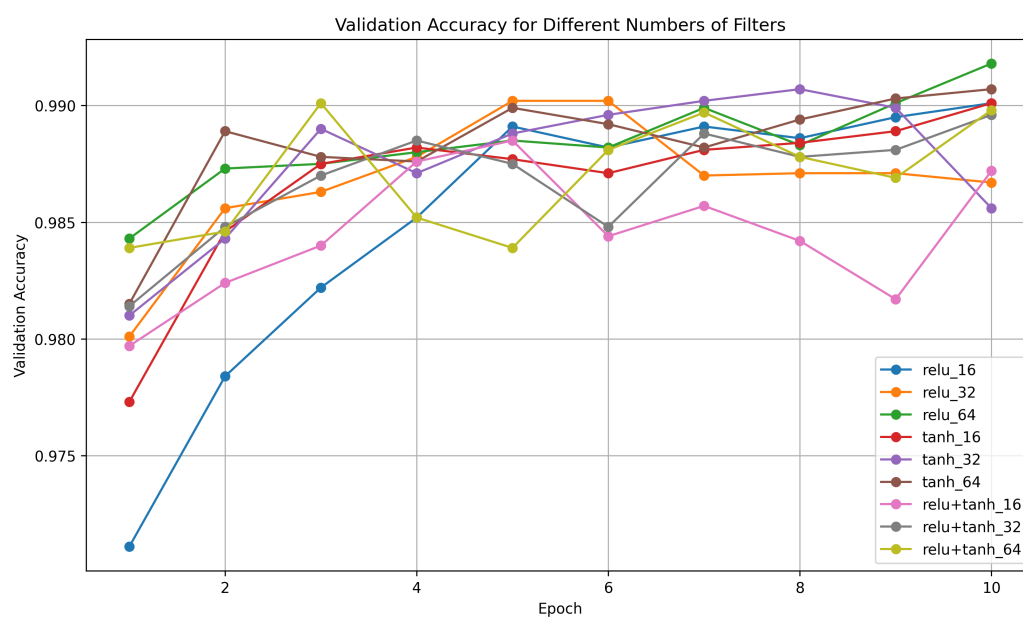


Abbildung 4: Graphischer Vergleich der Filterzahlen

Results for Different Numbers of Filters (Best Model Highlighted)

activation	base_filters	conv_blocks	val_loss	val_accuracy	params
relu	64	3	0.0391	0.9918	665,994
tanh	64	3	0.0372	0.9907	665,994
relu	16	3	0.0433	0.9901	98,442
tanh	16	3	0.0415	0.9901	98,442
relu+tanh	64	3	0.0523	0.9898	665,994
relu+tanh	32	3	0.0542	0.9896	241,546
relu+tanh	16	3	0.0581	0.9872	98,442
relu	32	3	0.0441	0.9867	241,546
tanh	32	3	0.0535	0.9856	241,546

Tabelle 6: Tabellarische Ergebnisse der Filterzahlen

07.03.2026: Weitere Analyse der Anzahl von Convolutional Layers, Filter und Activation Maps

Während des Treffens vom 05.03.2026 kam es zu Unterschieden bezüglich der Ergebnisse von Convolutional Layers. Diese sollten weiter analysiert werden.

- Visualisierung der Activation Maps und Analyse der Convolutional Stats:

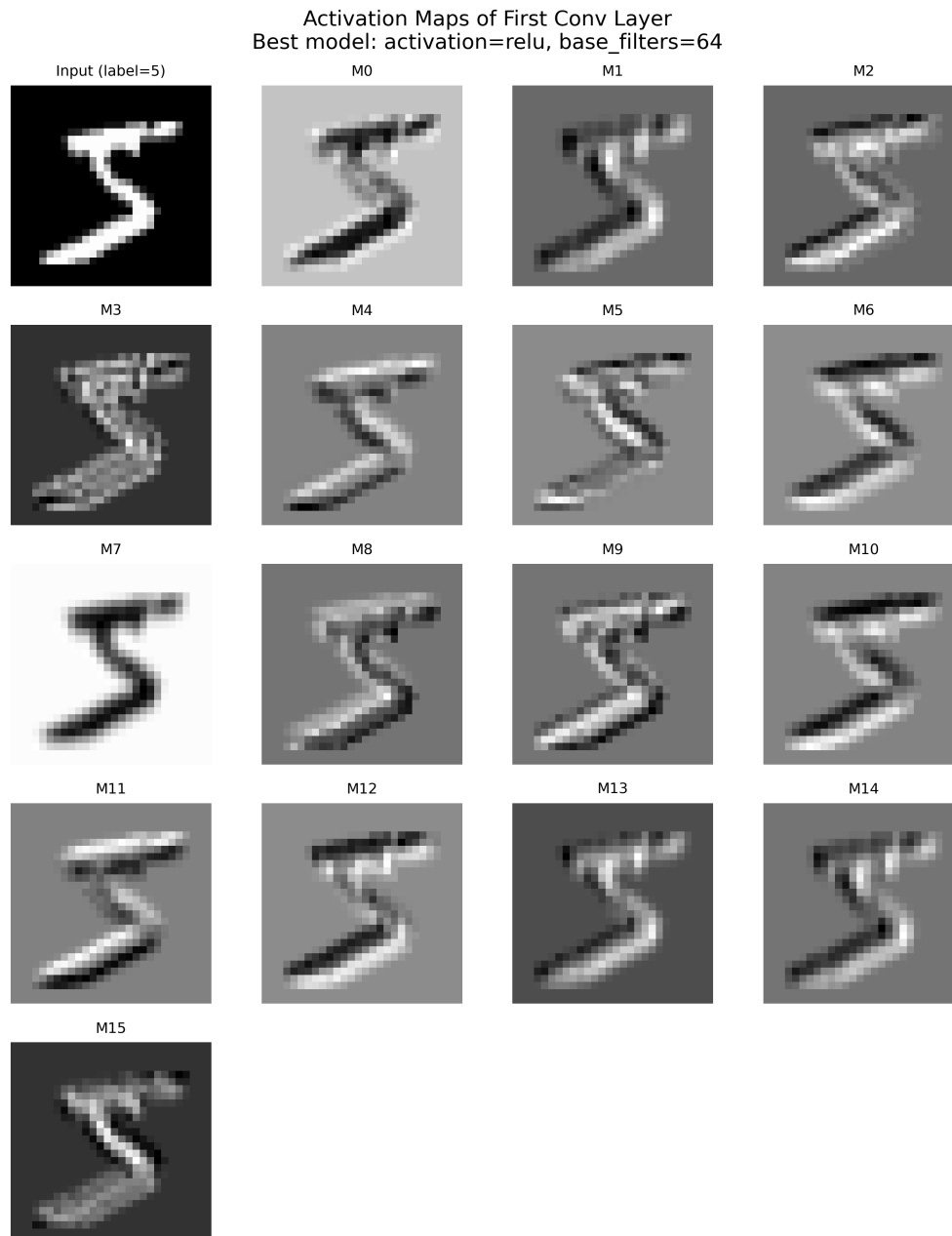


Abbildung 5: Activation Maps der ersten Convolution-Schicht

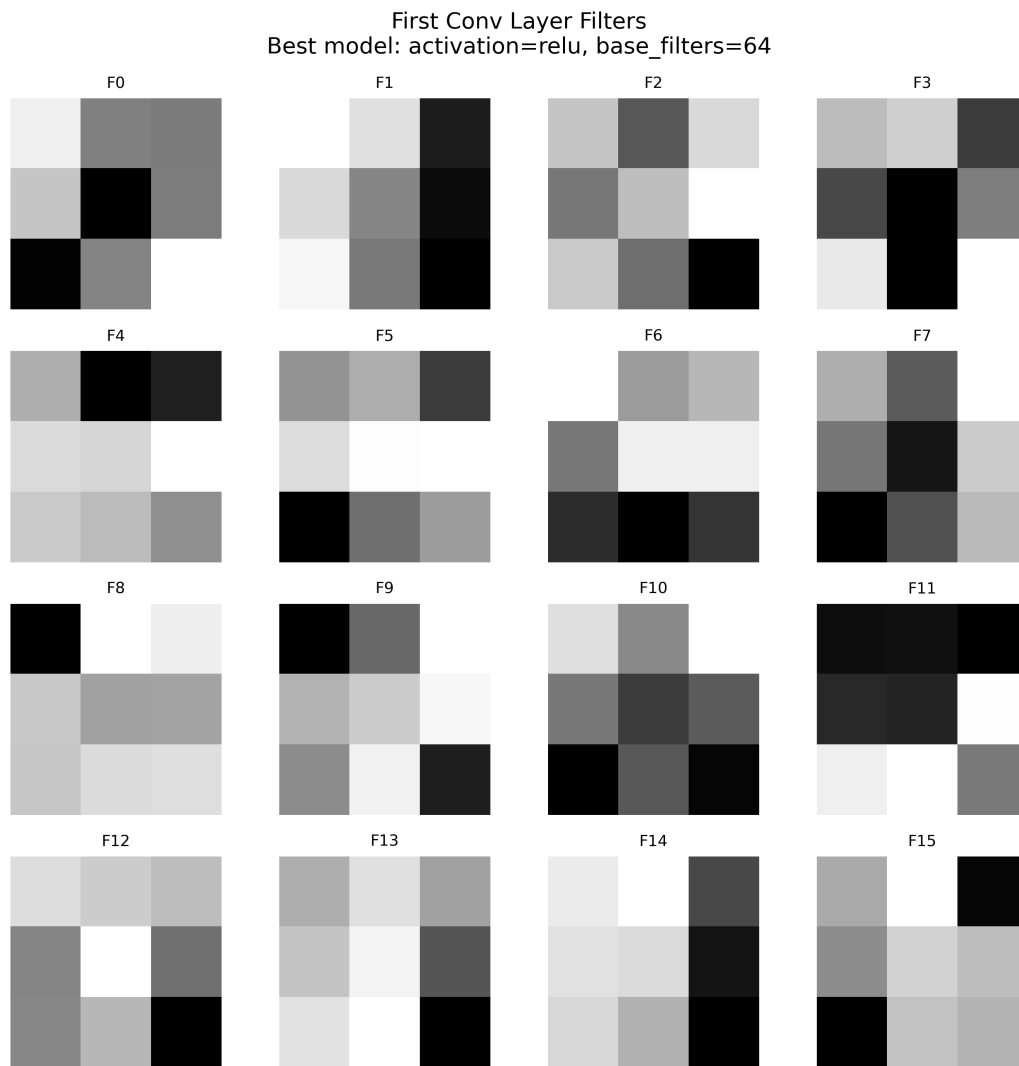


Abbildung 6: Filter der ersten Convolution-Schicht

Convolution Filter Statistics of the Best Model

layer	w_mean	w_std	w_min	w_max	n_filters	kernel
conv_1	-0.0032	0.1099	-0.3848	0.2615	64	(3, 3)
conv_2	-0.0124	0.0722	-0.3931	0.3439	128	(3, 3)
conv_3	-0.0114	0.0522	-0.3320	0.2454	256	(3, 3)

Tabelle 7: Statistische Analyse der Convolution-Filter

- Ergebnisse:

- Einfache Kodierung der Muster
- Anpassung der Gewichte

Beeinflusst die Kodierung in der ersten Schicht den weiteren Verlauf des Prozesses? Mit anderen Worten: Lässt sich die Kodierung verbessern? Wenn ja, wie?

10.03.2026: Weitere Hyperparameteroptimierung

Anzahl der Dense Layers, Learning Rate, Dropout Rate

- Fragestellung und Vorgehen:

- Zu den Dense Layers

Da die Dense Layers für die Entscheidungsklassifikation zuständig sind, und zwar nach dem Flattening der Feature Maps, kam ich zur Idee, ihre Anzahl zu erhöhen. Das Ziel war zu untersuchen, ob eine größere Zahl von Dense Layers zu einer besseren Klassifikation führen würde. Genutzt wurden `n_dense_layers_list = (1,2,3)`.

- Zur Learning Rate

Getestet wurde `learning_rate_list = (1e-2,1e-3,1e-4)`.

- Dropout

Die Dropout-Rate ist ebenfalls ein Hyperparameter, da sie vor dem Training festgelegt wird und die Regularisierung des Modells beeinflusst. Eine höhere Dropout-Rate kann Overfitting reduzieren, während eine zu starke Regularisierung die Modelleleistung verschlechtern kann.

Mit der Methode Grid Search wurden zusammen getestet:

- Filters: {16, 32, 64}
- Activation: {relu, tanh, relu+tanh}
- Convolutional Blocks: {1, 2, 3}
- Dense Layers: {1, 2, 3}
- Dropout: {0.00, 0.25, 0.50}
- Learning Rate: {1e-2, 1e-3, 1e-4}

Ergebnisse:

Die davon 10 besten Modelle wurden beibehalten und sind in der folgenden Tabelle zu finden.

Top 10 CNN Models by Validation Accuracy										
filters	activation	conv_blocks	dense_layers	dropout	learning_rate	train_loss	train_accuracy	val_loss	val_accuracy	params
64	relu	3	2	0.25	0.001	0.0087	0.9978	0.0455	0.9921	675370
32	tanh	3	2	0.5	0.001	0.0076	0.9974	0.041	0.9914	249162
32	relu+tanh	3	2	0.25	0.001	0.0081	0.9978	0.0349	0.9911	249162
32	relu	2	1	0.25	0.001	0.0046	0.9986	0.0341	0.991	673610
64	relu+tanh	3	3	0.5	0.001	0.0088	0.9972	0.0472	0.9907	673610
64	relu	3	2	0.5	0.001	0.0101	0.9968	0.0379	0.9907	673610
32	relu	3	3	0.25	0.001	0.0144	0.996	0.0421	0.9906	250922
32	relu	3	3	0.0	0.001	0.0095	0.997	0.0415	0.9904	250922
32	relu+tanh	3	3	0.25	0.001	0.0134	0.9961	0.0464	0.9904	250922
64	relu	3	3	0.5	0.001	0.0159	0.996	0.0609	0.9904	675370

Tabelle 8: Top 10 Modelle der Grid Search

Das Modell mit ReLU hat die beste *val_accuracy*.

Das Modell mit ReLU+Tanh hat eine ziemlich gute *val_accuracy* und verlangt weniger Parameters.

Die Learning Rates 10^{-2} und 10^{-4} tauchen überhaupt nicht in den 10 besten Modellen auf, was hindeuten kann, dass extrem große oder kleine Werte der Learning Rate nicht optimal sind.

Mich hat ebenfalls interessiert, welche Ziffern falsch klassifiziert wurden. Deswegen wurde die Confusion Matrix erzeugt.

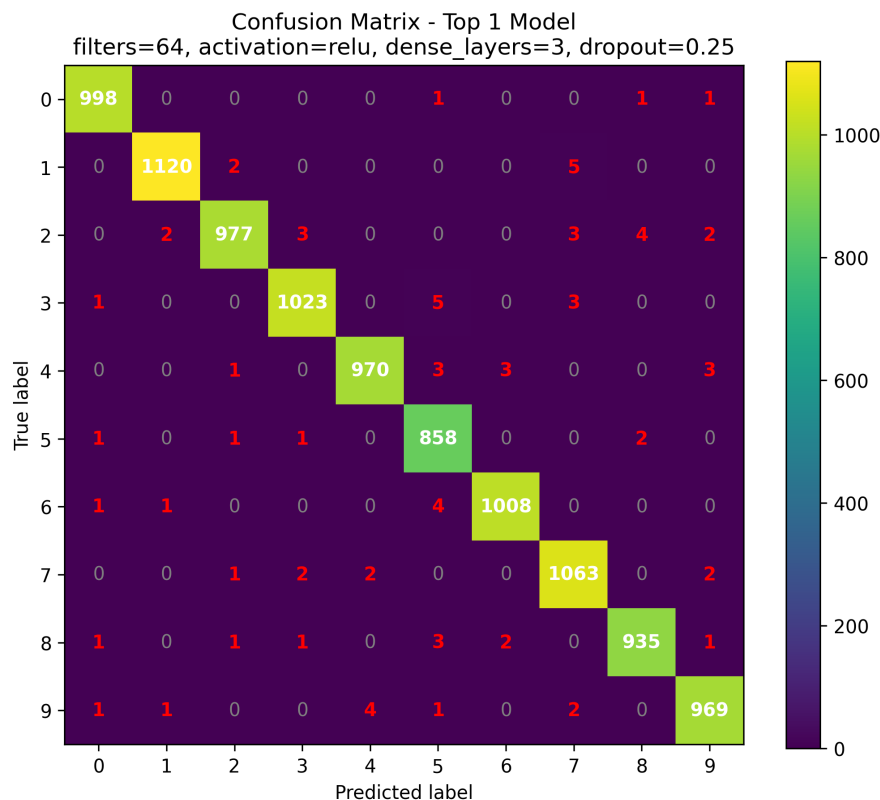


Abbildung 7: Confusion Matrix des besten Modells

- Schwierigkeit:

Die Laufzeit war zu groß (mehr als 20 Stunden).

13.03.2026: Einfluss der Dropout und Anwendung des Keras Tuner

Dropout

- Begründung:

Während der Sitzung vom 12.03.2026 wurde die Frage gestellt, ob es überhaupt nötig ist, den Dropout in einem Modell zu nutzen. Deswegen habe ich mich entschieden, den Dropout separat zu testen, dies nur mit dem besten Modell der vorherigen Untersuchung.

- **Ergebnisse:** Dropout hilft, eine erhöhte Modellleistung zu haben, insbesondere bei vernünftigen Werten.

Dropout Comparison Results

dropout	learning_rate	train_loss	train_accuracy	val_loss	val_accuracy	params
0.2500	0.0010	0.0069	0.9979	0.0363	0.9910	673610
0.5000	0.0010	0.0191	0.9949	0.0458	0.9904	673610
0.7500	0.0010	0.0221	0.9956	0.0654	0.9899	673610
0.0000	0.0010	0.0114	0.9964	0.0399	0.9890	673610

Tabelle 9: Tabellarische Ergebnisse des Dropout-Tests

Tuning

Wegen des großen Zeitaufwands der durchgeführten Grid Search wurde während dieses Treffens ebenfalls vorgeschlagen, das Tuning in Anwendung zu bringen, um zu sehen, was für Ergebnisse erzielt werden könnten. Dabei wurden genau die gleichen Hyperparameterlisten genutzt wie bei der Grid Search, nämlich:

- Filters: {16, 32, 64}
- Activation: {relu, tanh, relu+tanh}
- Convolutional Blocks: {1, 2, 3}
- Dense Layers: {1, 2, 3}
- Dropout: {0.00, 0.25, 0.50}
- Learning Rate: {1e-2, 1e-3, 1e-4}

- **Ergebnisse:** Die 10 besten Modelle wurden ebenfalls tabellarisch dokumentiert und sie unterscheiden sich ganz von denen der Grid Search.

Top 10 Models from Keras Tuner

rank	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	train_loss	train_accuracy	val_loss	val_accuracy	params
1	64	tanh	3	1	0.5000	0.0010	0.0091	0.9976	0.0359	0.9902	665994
2	64	relu	3	2	0.2500	0.0010	0.0143	0.9955	0.0369	0.9900	673610
3	16	relu	2	1	0.2500	0.0010	0.0240	0.9925	0.0438	0.9879	206922
4	32	relu	3	3	0.0000	0.0010	0.0184	0.9944	0.0395	0.9878	250922
5	16	tanh	2	2	0.0000	0.0010	0.0165	0.9953	0.0492	0.9856	214538
6	16	relu	3	1	0.2500	0.0100	0.0352	0.9890	0.0524	0.9851	98442
7	16	relu	1	2	0.2500	0.0010	0.0243	0.9932	0.0537	0.9843	410602
8	16	relu	2	3	0.2500	0.0100	0.0404	0.9896	0.0799	0.9830	216298
9	64	relu+tanh	3	3	0.5000	0.0001	0.0442	0.9863	0.0611	0.9824	675370
10	32	relu+tanh	2	1	0.2500	0.0100	0.0552	0.9836	0.0774	0.9794	421642

Tabelle 10: Top 10 Modelle des Keras Tuners

Das beste Modell hier hat 99,02% als Validierungsscore, während das zehnte Modell der Grid Search schon 99,04% hat, also mehr. Das heißt, das beste Modell hier würde nicht zu den Top 10 der Grid Search gehören. Man kann davon ausgehen, dass das beste Modell des Tuning nur ein lokales Maximum ist. Dies könnte daran liegen, dass der Tuner eigentlich nicht alle Konfigurationen testet.

17.03.2026: Erweiterung zur Batch Normalization, Batch Size, Dense Units und Optimizers

Die Zusammensetzung mit diesen Aspekten kommt aus der Sitzung vom 12.03.2026.

Batch Normalization

- Begründung

Die Batch Normalization ist eine Technik zur Stabilisierung und Beschleunigung des Trainings neuronaler Netze. Dabei werden die Eingaben jeder Schicht normalisiert (Mittelwert ≈ 0 , Varianz ≈ 1), wodurch das sogenannte „Internal Covariate Shift“ reduziert wird. Sie wurde nach der Hyperparameteroptimierung ebenso getestet, um zu prüfen, ob eine Stabilisierung der Aktivierungen während des Trainings die Modellleistung verbessert.

- Vorgehen

Dafür wurden die zwei besten Modelle der Grid Search übernommen und lediglich Batch Normalization ergänzt. Eine erneute Trainingsphase war notwendig, da Batch Normalization die Verteilung der Aktivierungen während des Trainings verändert. Dadurch werden die Gewichte des Modells neu angepasst, sodass ein zuvor trainiertes Modell ohne erneutes Training nicht korrekt mit Batch Normalization funktionieren würde.

- Ergebnisse

Batch Normalization Comparison

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	batchnorm	train_loss	train_accuracy	val_loss	val_accuracy	params
model_r	64	relu	3	2	0.2500	0.0010	True	0.0103	0.9968	0.0320	0.9907	676170
model_r	64	relu	3	2	0.2500	0.0010	False	0.0148	0.9954	0.0345	0.9899	673610
model_rt	32	relu+tanh	3	2	0.2500	0.0010	False	0.0115	0.9965	0.0449	0.9890	249162
model_rt	32	relu+tanh	3	2	0.2500	0.0010	True	0.0165	0.9949	0.0411	0.9877	250826

Tabelle 11: Ergebnisse mit Batch Normalization

Batch Normalization wurde getestet, zeigte jedoch keinen signifikanten Einfluss auf die Modellleistung. Tatsächlich ist zwar das beste Modell mit `batchnorm = True`, aber sowohl die *train_accuracy* als auch die *val_accuracy* im Vergleich zu denen der Grid Search haben sich verschlechtert. Ist es auf die Batch Normalization zurückzuführen oder eher auf die Tatsache, dass die Trainingsdaten beim neuen Training unterschiedlich waren? Anders gesagt: Hätten wir bessere Ergebnisse gehabt, falls die Modelle wiederum auf denselben Trainingsdaten der Grid Search trainiert worden wären?

Batch Size

Zunächst wurde mit der Standardwahl `batch_size = 128` gearbeitet. Im weiteren Verlauf stellte sich die Frage, ob die Batch-Größe selbst einen Einfluss auf die Modellleistung hat. Daher wurde dieser Trainingsparameter systematisch variiert (32, 64, 128). Dabei zeigte sich, dass kleinere Batch-Größen tendenziell zu einer besseren Generalisierung führen können, nehmen aber viel Zeit in Anspruch.

Batch Size Comparison (Best CNN Model)

batch_size	train_loss	train_accuracy	val_loss	val_accuracy	time_sec
32.0000	0.0055	0.9982	0.0492	0.9913	1411.1919
64.0000	0.0064	0.9980	0.0421	0.9906	1145.7784
128.0000	0.0054	0.9979	0.0420	0.9898	1062.4118
256.0000	0.0062	0.9980	0.0370	0.9894	1130.6644

Tabelle 12: Tabellarische Ergebnisse zur Batch Size

Dense Units

- Begründung/Vorgehen

Die Kapazität der Dense-Schicht wurde untersucht. Dabei wurde die Anzahl der Neuronen variiert, um zu analysieren, wie sich die Modellkomplexität im Klassifikationsbereich auf die Performance auswirkt.

Das model_1 (Tabelle 15) wurde genutzt und getestet wurde `dense_units_list = {64, 128, 256}`.

- Ergebnisse:

Results for Different Dense Units

dense_units	train_loss	train_accuracy	val_loss	val_accuracy
128	0.0064	0.9978	0.0346	0.9918
256	0.0086	0.9973	0.0440	0.9893
64	0.0087	0.9973	0.0417	0.9891

Tabelle 13: Tabellarische Ergebnisse zu den Dense Units

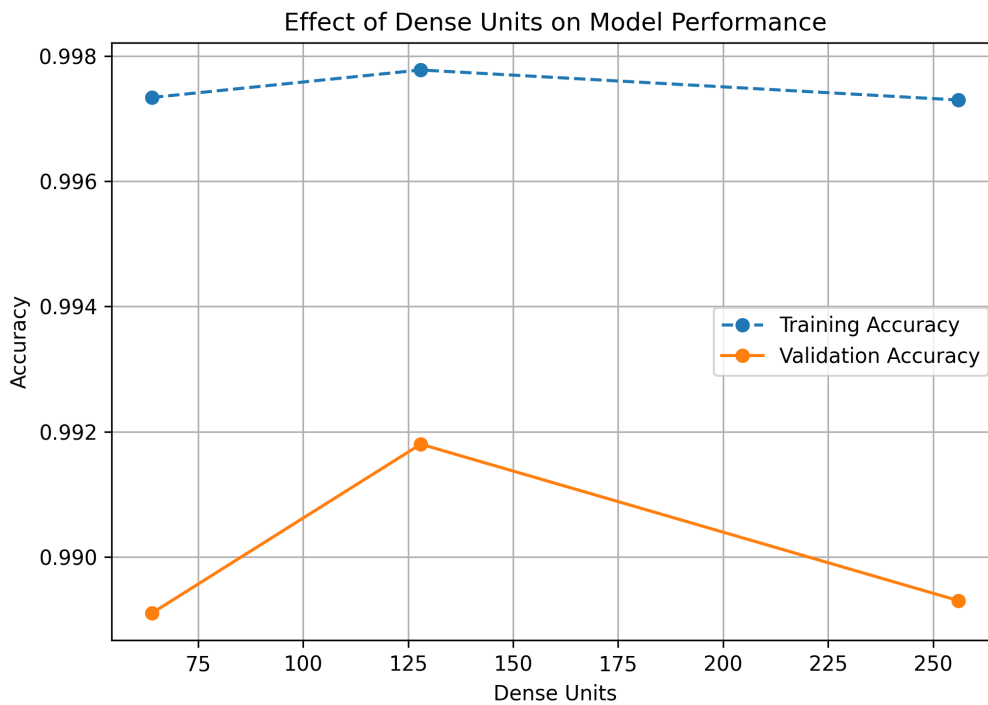


Abbildung 8: Grafische Darstellung der Dense-Units-Ergebnisse

Die Größe 128 scheint die besten Ergebnisse zu haben.

Optimizer Adam vs SGD

- Fragestellung:

Bisher wurde ausschließlich der Adam-Optimizer verwendet. Es stellte sich die Frage, ob die Wahl des Optimizers selbst einen Einfluss auf die Modellleistung hat. Infolgedessen wurde ein Vergleich zwischen Adam und SGD (Stochastic Gradient Descent) durchgeführt.

Optimizer Comparison: Adam vs SGD

optimizer	train_loss	train_accuracy	val_loss	val_accuracy
adam	0.0071	0.9976	0.0416	0.9906
sgd	0.0083	0.9982	0.0370	0.9896

Tabelle 14: Tabellarischer Vergleich zwischen Adam und SGD

Der Optimizer SGD scheint beim Training eine bessere Accuracy zu haben, diese ist jedoch bei der Validierung abgestiegen. Die vernünftige Entscheidung wäre daher, beim Adam Optimizer zu bleiben.

Ist es aber möglich, zwei Optimizer zu kombinieren?

Speicherung der 4 besten Modelle

Von den bisher besten Modellen wurden die 4 besten gespeichert zu Weiterverlauf.

Top 4 Selected Models (Validation Results)

model_name	filters	activation	conv_blocks	dense_layers	dropout	batch_size	dense_units	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.25	128	128	0.001	0.022817	0.9946
model_4	32	relu	2	1	0.25	128	128	0.001	0.029877	0.9928
model_2	64	tanh	3	2	0.50	128	128	0.001	0.030876	0.9924
model_3	32	relu+tanh	3	2	0.25	128	128	0.001	0.040122	0.9894

Tabelle 15: Speicherung der vier besten Modelle

20.03.2026: Anwendung und Training auf den modifizierten Daten

Begründung des Abschnitts:

Nach dem Treffen vom 19.03.2026 wurde entschieden, dass Dung und ich unsere Modelle auf modifizierten Daten validieren. Ziel war es, die Reaktion der Modelle gegenüber modifizierten Daten zu untersuchen.

Anwendung:

Für jedes Rauschen sind die Ergebnisse in einer Tabelle dokumentiert, und aus Lesbarkeitsgründen werden nur bei manchen Rauschen die Confusion Matrix des effizientesten Modells gezeigt.

Gaussisches Rauschen

Validation Results on Gaussian Noise

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0287	0.9918
model_4	32	relu	2	1	0.2500	0.0010	0.0348	0.9902
model_2	64	tanh	3	2	0.5000	0.0010	0.0422	0.9880
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.0553	0.9850

Tabelle 16: Ergebnisse für gaußsches Rauschen

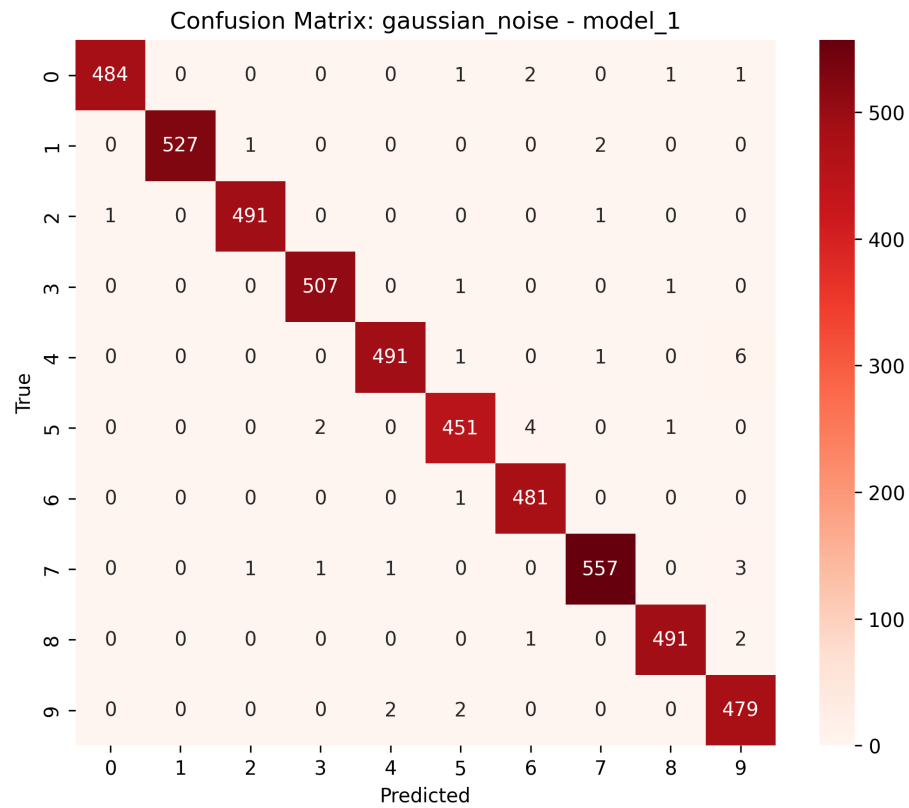


Abbildung 9: Confusion Matrix des besten Modells bei gaußischem Rauschen

Rotierung

Validation Results on Rotation

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0518	0.9870
model_2	64	tanh	3	2	0.5000	0.0010	0.0693	0.9836
model_4	32	relu	2	1	0.2500	0.0010	0.0687	0.9814
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.0781	0.9764

Tabelle 17: Ergebnisse für Rotierung

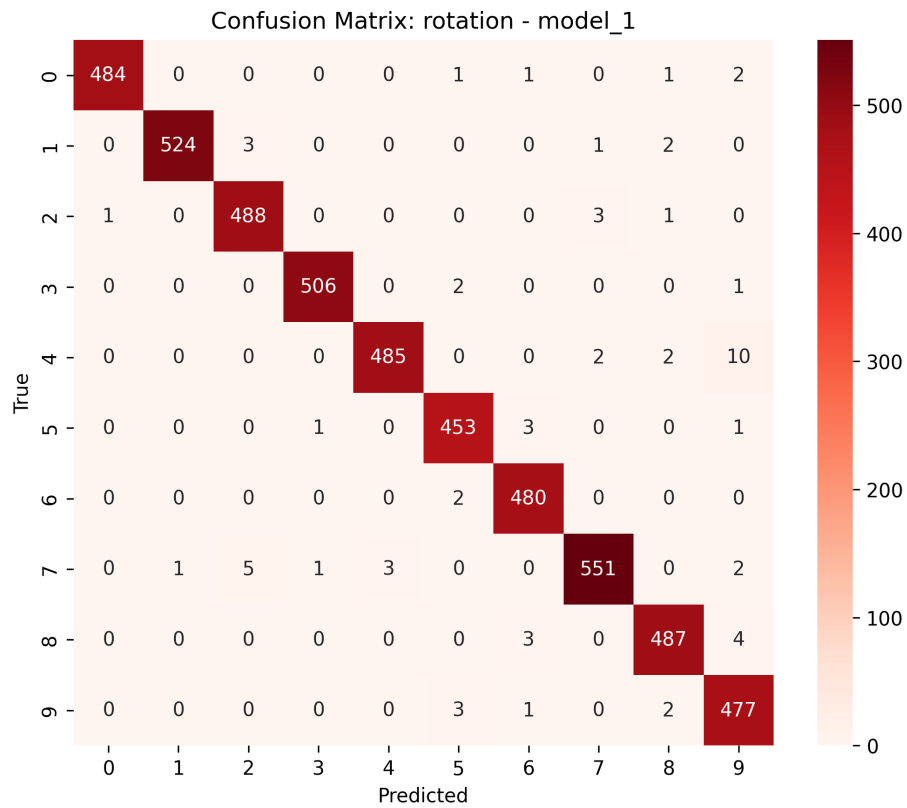


Abbildung 10: Confusion Matrix bei Rotierung

Pepper und Salz

Validation Results on Salt Pepper

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0343	0.9890
model_2	64	tanh	3	2	0.5000	0.0010	0.0673	0.9812
model_4	32	relu	2	1	0.2500	0.0010	0.0668	0.9800
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.1133	0.9632

Tabelle 18: Ergebnisse für Pepper-und-Salz-Rauschen

Dickere Zahlen

Validation Results on Thicker Digits

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0292	0.9930
model_4	32	relu	2	1	0.2500	0.0010	0.0378	0.9906
model_2	64	tanh	3	2	0.5000	0.0010	0.0389	0.9894
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.0470	0.9892

Tabelle 19: Ergebnisse für dickere Zahlen

Verschiebung

Validation Results on Shift Xy

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.2039	0.9448
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.3118	0.9128
model_2	64	tanh	3	2	0.5000	0.0010	0.3944	0.8992
model_4	32	relu	2	1	0.2500	0.0010	0.6630	0.8474

Tabelle 20: Ergebnisse für Verschiebung

Speckled Rauschen

Validation Results on Speckled Noise

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0227	0.9942
model_4	32	relu	2	1	0.2500	0.0010	0.0314	0.9920
model_2	64	tanh	3	2	0.5000	0.0010	0.0341	0.9914
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.0396	0.9888

Tabelle 21: Ergebnisse für Speckled Rauschen

Kombination Gaussian + Pepper and Salt

Validation Results on Gaussian Plus Saltpepper

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_1	64	relu	3	2	0.2500	0.0010	0.0498	0.9862
model_2	64	tanh	3	2	0.5000	0.0010	0.0781	0.9762
model_4	32	relu	2	1	0.2500	0.0010	0.0864	0.9722
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.2210	0.9262

Tabelle 22: Ergebnisse für kombinierte Störung

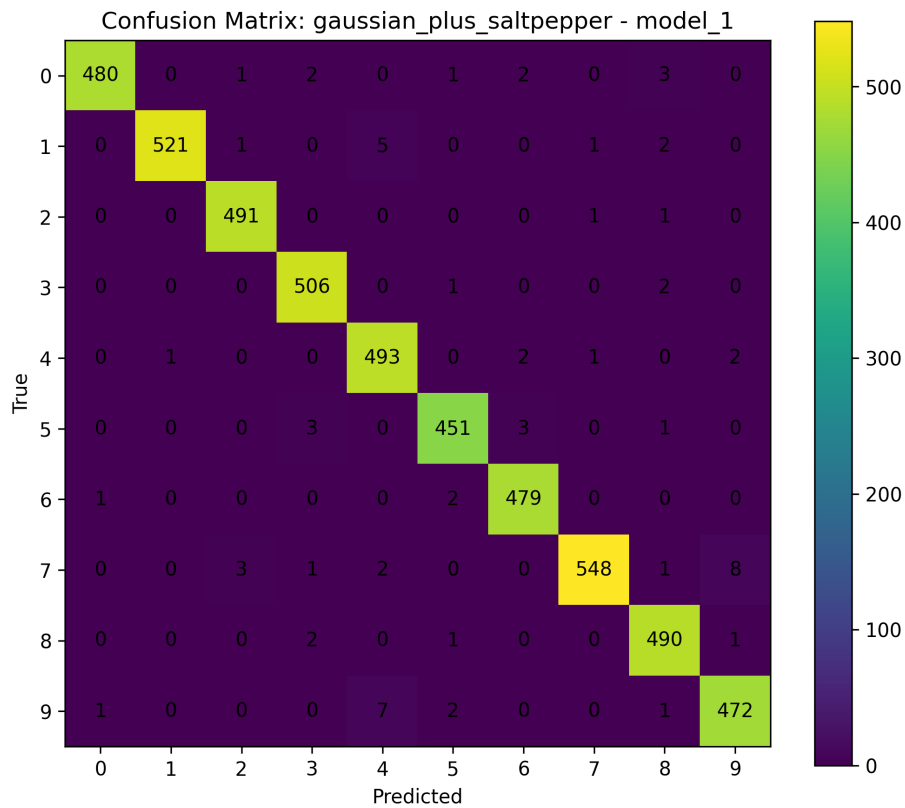


Abbildung 11: Confusion Matrix bei kombinierter Störung

Die Modelle reagierten eher besser als gedacht auf die Mischung von Rauschen. Liegt es daran, dass sie schon vorher auf den einzelnen Modifikationen validiert wurden?

Okklusion

Im Rahmen dieses Schrittes wurden die Eingabebilder gezielt verändert, indem Teile der Ziffern verdeckt (okkludiert) wurden. Dadurch fehlen dem Modell bewusst wichtige Informationen, die ursprünglich zur Klassifikation zur Verfügung standen.

Validation Results on Occlusion

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	val_loss	val_accuracy
model_4	32	relu	2	1	0.2500	0.0010	0.2691	0.9274
model_1	64	relu	3	2	0.2500	0.0010	0.3324	0.9244
model_2	64	tanh	3	2	0.5000	0.0010	0.2882	0.9244
model_3	32	relu+tanh	3	2	0.2500	0.0010	0.3377	0.9100

Tabelle 23: Ergebnisse bei Okklusion

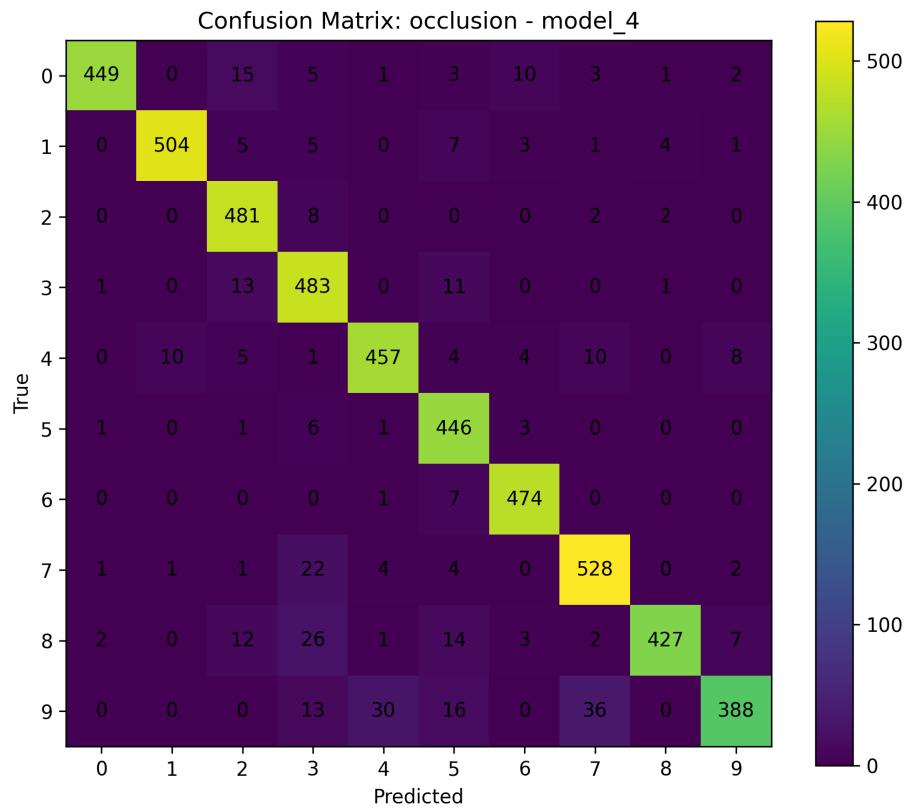


Abbildung 12: Confusion Matrix bei Okklusion

Die Ergebnisse bei dieser Modifikation waren die schlechtesten.

Retraining

Die Modelle wurden wieder auf normalen + modifizierten Daten trainiert und validiert. Folgende Abbildungen zeigen die Robustheit und Konfidenz vor und nach dem Retraining.

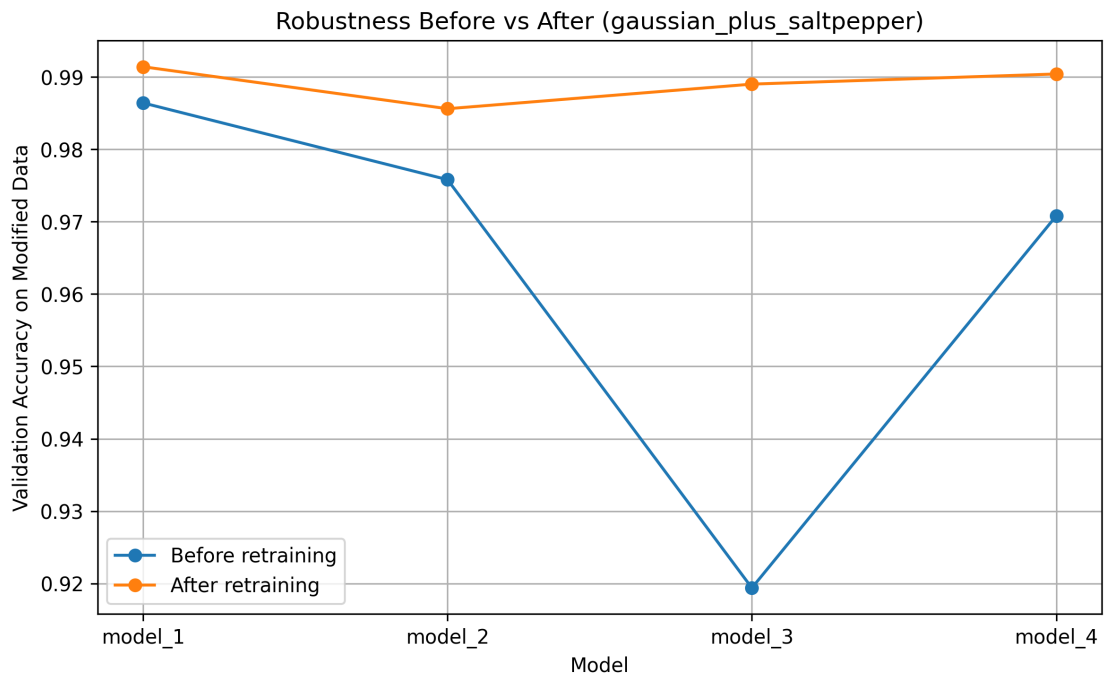


Abbildung 13: Robustheit vor und nach dem Retraining

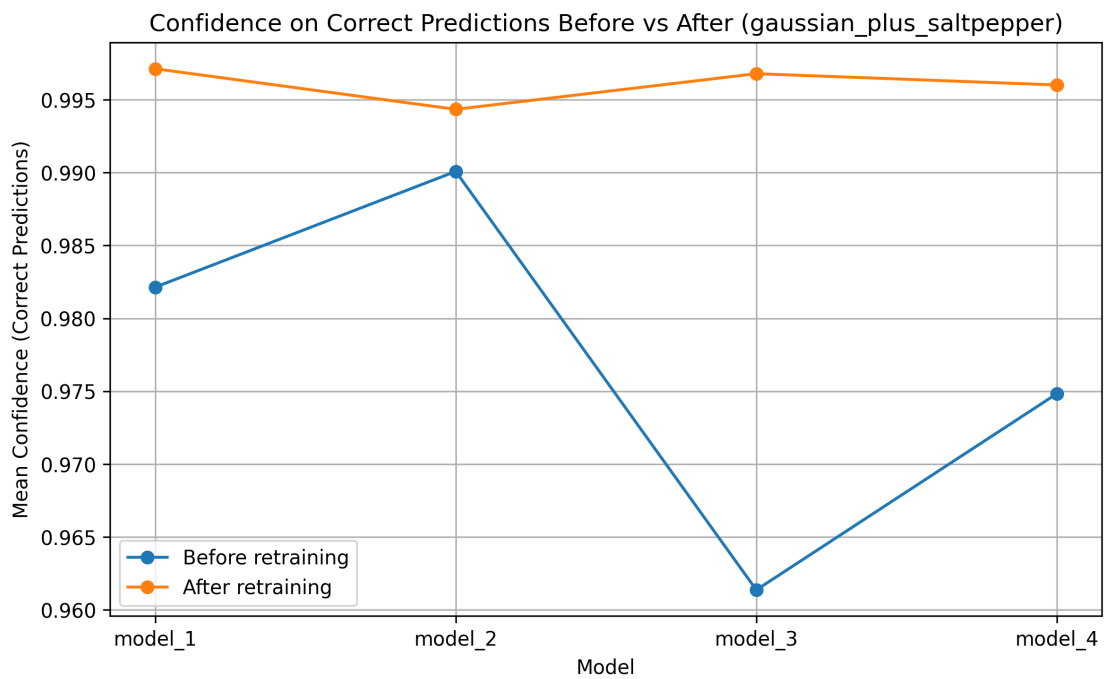


Abbildung 14: Konfidenz richtige Vorhersagen vor und nach dem Retraining

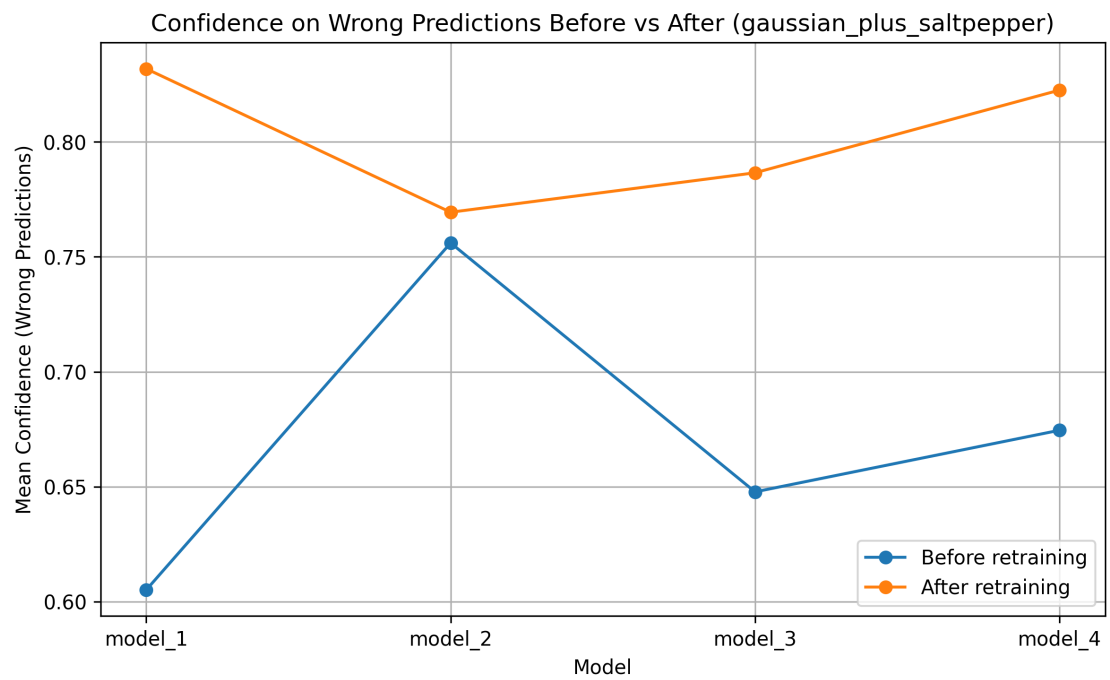


Abbildung 15: Konfidenz falsche Vorhersagen vor und nach dem Retraining

25.03.2026: Evaluierung auf den Testdaten

Test Results of Selected Models

model_name	filters	activation	conv_blocks	dense_layers	dropout	learning_rate	test_loss	test_accuracy	mean_confidence
model_1	64	relu	3	2	0.25	0.001	0.0246	0.9928	0.9959
model_4	32	relu	2	1	0.25	0.001	0.0285	0.9913	0.9933
model_2	64	tanh	3	2	0.5	0.001	0.0334	0.9907	0.9953
model_3	32	relu+tanh	3	2	0.25	0.001	0.0377	0.9882	0.9913

Tabelle 24: Testergebnisse auf den Testdaten

Quellen und Hilftmittel

Im Rahmen dieses Projekts wurden unterschiedliche Hilftmittel genutzt.

- Notebooks aus der Vorlesung
- [Keras Dokumentation](#)
- [TensorFlow Dokumentation](#)
- Generative KI unterstützend bei der Code-Erstellung

Beispiele von Prompts

- Passe den folgenden Code so an, dass mehrere Parameter auf einmal iteriert werden.
- Bei Ausführung meines Codes erhalte ich die folgende Fehlermeldung und finde die Fehlerquelle nicht. Kannst du helfen?